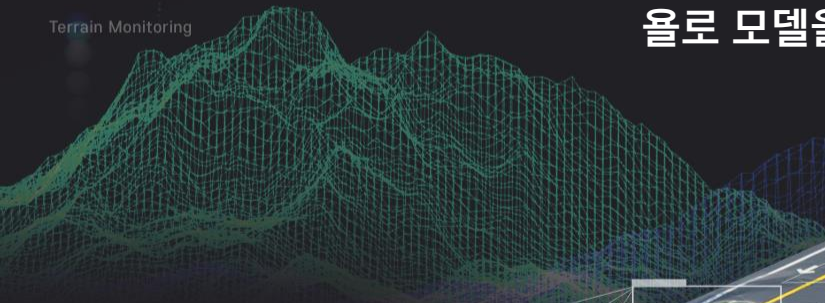


Image Recognition



Terrain Monitoring



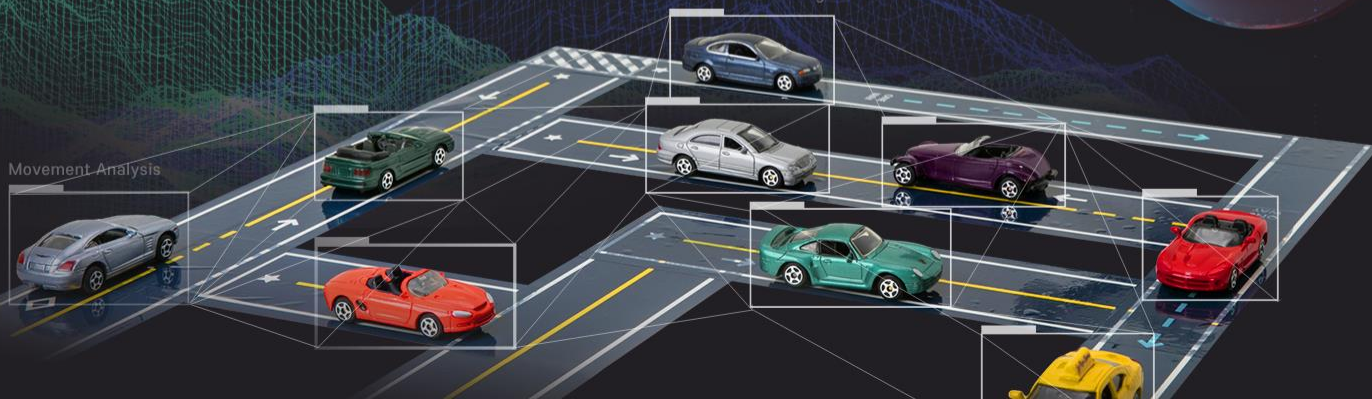
강원혁신플랫폼

컴퓨터 비전

올로 모델을 이용한 비디오 객체 인식

Autonomous Driving

Movement Analysis

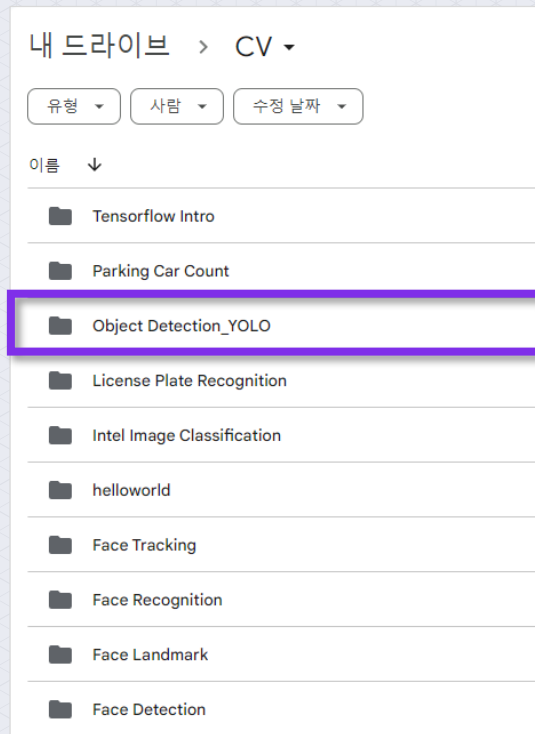


■ 비디오도 이미지에서와 동일한 방법으로 객체 검출을 할 수 있음

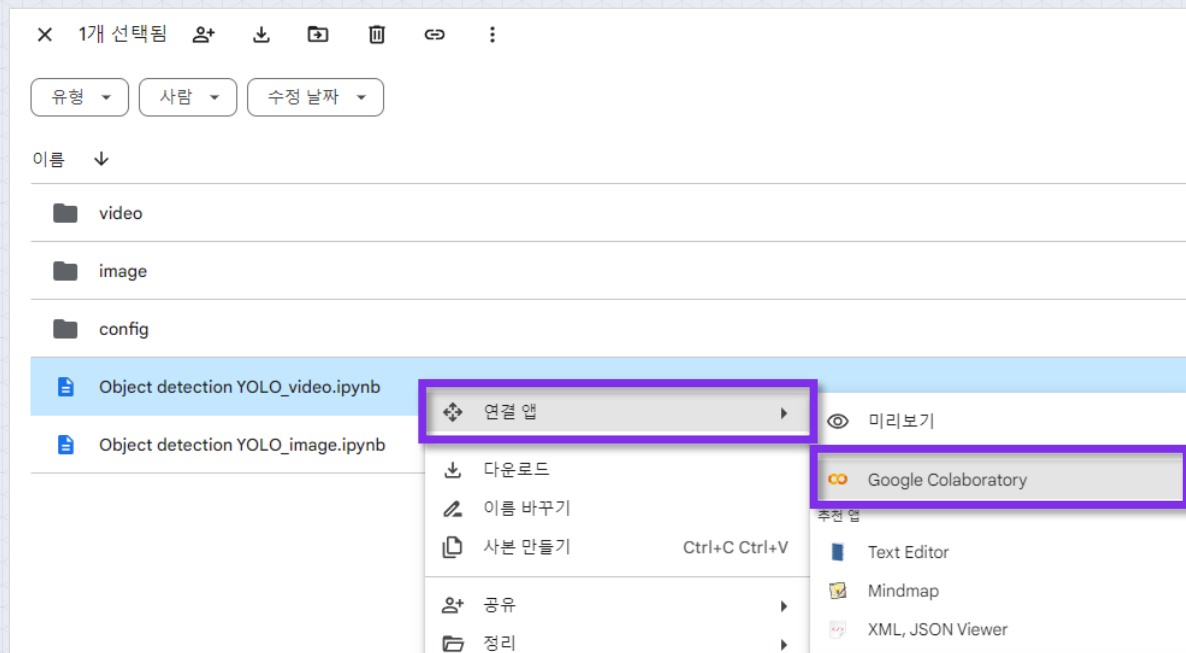
🔄 YOLOv3을 이용해 비디오에서 객체 검출을 해보자



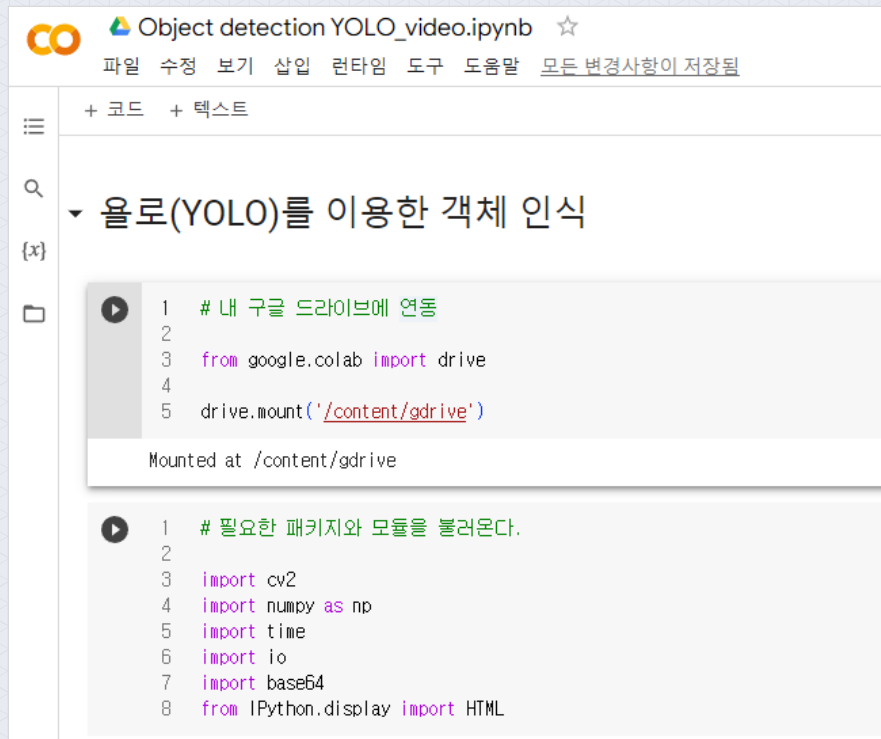
구글 드라이브 > CV > Object Detection_YOLO 폴더를 클릭



- 아래의 그림의 Object Detection_YOLO 폴더에서 Object detection YOLO_video.ipynb를 선택 후 오른쪽 마우스를 클릭하여 “연결 앱 > Google Colaboratory”을 클릭함



🔄 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것



Object detection YOLO_video.ipynb ☆

파일 수정 보기 삽입 런타임 도구 도움말 모든 변경사항이 저장됨

+ 코드 + 텍스트

☰ 🔍 {x} 📁

▼ 율로(YOLO)를 이용한 객체 인식

```
1 # 내 구글 드라이브에 연동
2
3 from google.colab import drive
4
5 drive.mount('/content/gdrive')
```

Mounted at /content/gdrive

```
1 # 필요한 패키지와 모듈을 불러온다.
2
3 import cv2
4 import numpy as np
5 import time
6 import io
7 import base64
8 from IPython.display import HTML
```

- ② 내 구글 드라이브에 연동
- ② 구글 Colab 노트북 화면에서 실행 버튼 클릭



- ② [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택하면 gdrive에 연동
- ② gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있음

구글 Colab 화면에서 내 구글 드라이브 연동

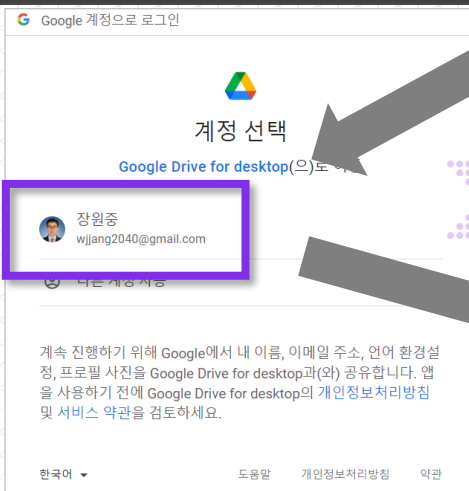
- [Google Drive에 연결] > [계정 선택] > [구글 드라이버 접근 허용]
- 아래 화면을 참고하여 **gdrive**에 연동

노트북에서 Google Drive 파일에 액세스하도록 허용하시겠습니까?

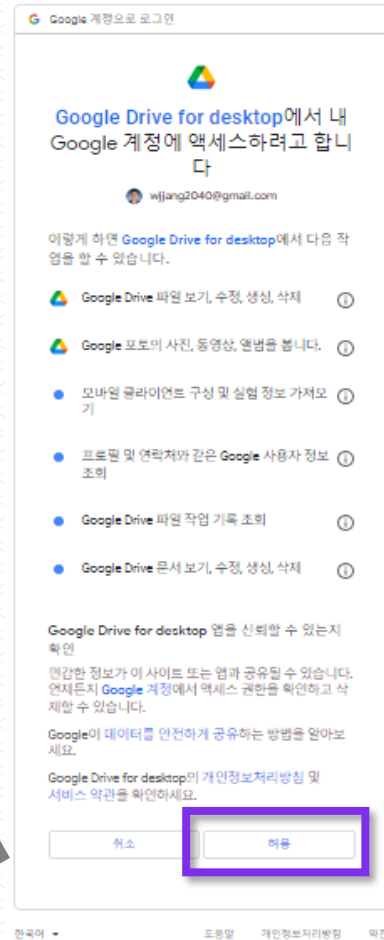
이 노트북에서 Google Drive 파일에 대한 액세스를 요청합니다. Google Drive에 대한 액세스 권한을 부여하면 노트북에서 실행되는 코드가 Google Drive의 파일을 수정할 수 있게 됩니다. 이 액세스를 허용하기 전에 노트북 코드를 검토하시기 바랍니다.

아니오

Google Drive에 연결



구글 드라이버 접근 허용



- 필요한 패키지와 모듈을 불러옴
- 구글 Colab 노트북 화면에서 실행 버튼 클릭

▶ # 필요한 패키지와 모듈을 불러온다.

```
import cv2
import numpy as np
import time
import io
import base64
from IPython.display import HTML
```

■ OpenCV의 버전을 확인해보자

- 🔄 구글 Colab 노트북 화면에서 실행 버튼 클릭

```
1  # OpenCV 버전 확인
2  print("OpenCV Version !!! ", cv2.__version__)
```

📄 OpenCV Version !!! 4.7.0

- 🔄 YOLOv3을 이용한 객체 검출은 OpenCV 3.4 이상 버전이 필요

■ 원본 비디오를 출력

```
1 # Detection 하기 전에 원본 비디오를 Display 한다.  
2  
3 video = io.open('gdrive/My Drive/CV/Object Detection_YOLO/video/street.mp4', 'r+b').read()  
4 encoded = base64.b64encode(video)  
5 HTML(data='''<video width="30%" controls>  
6 | | | | | <source src="data:video/mp4;base64,{0}" type="video/mp4"/>  
7 | | | | | </video>'''.format(encoded.decode('ascii')))
```



■ 비디오에서 사물 검출 및 검출결과 출력 파일로 저장하는 함수를 정의

```
def detectAndDisplay(frame):
    start_time = time.time()
    img = cv2.resize(frame, None, fx=0.9, fy=0.9)
    height, width, channels = img.shape

    # YOLOv3의 Detecting model 3가지(320×320, 416×416, 608×608)
    # 영상의 전처리
    blob = cv2.dnn.blobFromImage(img, 0.00392, (416, 416), (0, 0, 0), True, crop=False)

    net.setInput(blob)          # 네트워크에 전처리된 영상 입력 설정
    outs = net.forward(output_layers)  # 네트워크 순방향 추론

    # Showing informations on the screen
    class_ids = []              # detection 한 Class id를 저장하는 배열 정의
    confidences = []            # detection 한 Class 의 신뢰도(확률)를 저장하는 배열 정의
    boxes = []                  # detection 한 boxing 정보를 저장하는 배열 정의

    for out in outs:
        for detection in out:
            scores = detection[5:]
            class_id = np.argmax(scores)  # detection 한 Class id
            confidence = scores[class_id]  # detection 한 Class 의 신뢰도(확률)
            if confidence > min_confidence:
                # Object detected
                center_x = int(detection[0] * width)
                center_y = int(detection[1] * height)
                w = int(detection[2] * width)
                h = int(detection[3] * height)

                # Rectangle coordinates
                x = int(center_x - w / 2)
                y = int(center_y - h / 2)

                boxes.append([x, y, w, h])  # boxing 정보를 boxes 배열에 저장
                confidences.append(float(confidence))  # 신뢰도(확률)을 confidences 배열에 저장
                class_ids.append(class_id)  # Class id 를 class_ids 배열에 저장

    # apply non-max suppression
    # 박스안에 박스(노이즈)를 하나로 만들어 준다.
    indexes = cv2.dnn.NMSBoxes(boxes, confidences, min_confidence, nms_threshold)
```

```
font = cv2.FONT_HERSHEY_PLAIN
for i in range(len(boxes)):
    if i in indexes:  # 노이즈가 제거된 박스만 표시해 준다.
        x, y, w, h = boxes[i]

        # Class 이름, 신뢰도(확률) 표시
        label = "{}: {:.2f}".format(classes[class_ids[i]], confidences[i]*100)
        print(i, label)
        color = colors[i]
        cv2.rectangle(img, (x, y), (x + w, y + h), color, 1)
        cv2.rectangle(img, (x, y - 20), (x + w, y), color, -1)
        cv2.putText(img, label, (x + 5, y - 5), font, 1, (255, 255, 255), 1)

process_time = time.time() - start_time
global elapsed_time
elapsed_time += process_time  # 총 경과시간 누적
print("=== A frame took {:.3f} seconds".format(process_time))

# video 를 disk 에 output 하기 위해 writer 를 초기화한다.
global writer
if writer is None and output_name is not None:
    fourcc = cv2.VideoWriter_fourcc(*"MJPG")
    writer = cv2.VideoWriter(output_name, fourcc, 30,
                             (img.shape[1], img.shape[0]), True)

# disk 에 frame 을 write 합니다.
if writer is not None:
    writer.write(img)
```

■ MS COCO 데이터셋에 사용된 80가지 부류 이름을 읽어와서 배열로 저장

🔄 'coco.names' 파일에 들어 있는 클래스 이름의 공백을 제거하여 classes 리스트에 넣음

```
1 # detection 할 Object(Class) list 배열을 정의
2 classes = []
3
4 # 80개의 Object(class)를 구분할 수 있는 Object의 이름을 classes 배열에 넣어준다.
5 with open("gdrive/My Drive/CV/Object Detection_YOLO/config/coco.names", "r") as f:
6     # 클래스 이름의 공백을 제거하여 classes 리스트에 넣는다.
7     classes = [line.strip() for line in f.readlines()]
8 print("classes len:", len(classes))           # classes len: 80
9 print("classes:", classes)                   # classes: ['person', 'bicycle', 'car', 'motorbike', ...]
```

classes len: 80

classes: ['person', 'bicycle', 'car', 'motorbike', 'aeroplane', 'bus', 'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe', 'backpack', 'umbrella', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard', 'tennis racket', 'bottle', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake', 'chair', 'sofa', 'pottedplant', 'bed', 'diningtable', 'toilet', 'tvmonitor', 'laptop', 'mouse', 'remote', 'keyboard', 'cell phone', 'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush']

🔄 strip() : whitespace(띄어쓰기, 탭, 엔터)를 없애는 것으로 중간에 끼어있는 것은 없어지지 않음

■ 80개의 부류를 구분하기 위해 난수를 이용해 서로 다른 색깔을 설정함

- 80개 부류를 서로 다른 색깔로 구분하기 위한 난수를 생성하여 colors 변수에 할당
- » 아래 코드에서 0~255 사이의 랜덤 한 수로 (80, 3) 크기 형상으로 반환한 것을 볼 수 있음

`np.random.uniform(low=0.0, high=1.0, size=None)` :
random 모듈 안에 정의되어 있는 두 수 사이의 랜덤 한 소수를 리턴 시켜주는 함수

```
1 # Object 마다 컬러를 하나씩 다르게 지정
2 colors = np.random.uniform(0, 255, size=(len(classes), 3))
3 print("colors shape:", colors.shape) # colors shape: (80, 3)
4 print("colors:", colors[0:5])
```

```
colors shape: (80, 3)
colors: [[235.01093963 208.07098746 208.44801734]
 [219.59544969 137.54002927  64.0908944 ]
 [103.78250542  61.70933444 112.32695409]
 [105.3773565  182.90243816 207.76884854]
 [ 70.79845131 145.8140631 132.33892235]]
```

■ 신뢰도 점수와 NMS 임계값 설정

신뢰도=0.5, NMS=0.4로 임계값을 설정

```
1 # 하이퍼파라미터 설정
2
3 min_confidence = 0.5 # detection 으로 인정할 최소 확률(신뢰도) 지정 : 50%
4
5 # 비최대 억제(Non-Maximum Suppression, NMS) 알고리즘을 적용해 주위 바운딩 박스에 비해 최대를 유지한 것 인덱스 확인한다.
6 # - 박스안에 박스(노이즈)를 하나로 만들어 준다.
7
8 # 이 값이 클수록 box가 많이 사라진다(즉, 조금날 겹쳐도 NMS로 둘 중 하나를 삭제하기 때문)
9 nms_threshold = 0.4
```

YOLOv3.weights, YOLOv3.cfg

■ YOLOv3.weights, YOLOv3.cfg 파일을 불러와 YOLO 네트워크를 연결

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨

```
# Load YOLOv3
# 미리 학습된 덤러닝 파일을 OpenCV DNN 모듈로 YOLO 네트워크 연결
# 순전파(추론)만 가능하며 학습은 지원하지 않는다.

net = cv2.dnn.readNet("gdrive/My Drive/CV/Object Detection_YOLO/config/yolov3.weights",
                      "gdrive/My Drive/CV/Object Detection_YOLO/config/yolov3.cfg")
```

■ YOLOv3 네트워크의 모든 계층 이름을 출력

🔄 getLayerNames() 함수는 네트워크의 모든 레이어 이름을 가져옴

» YOLOv3 모델은 254개 계층으로 구성된 것을 알 수 있음



```
1 # YOLOv3 인공신경망 계층 이름을 출력
2
3 layer_names = net.getLayerNames()
4 print("layer_names len:", len(layer_names)) # layer_names len: 254
5 print("layer_names:", layer_names)         # layer_names: ('conv_0', 'bn_0', 'leaky_1', 'conv_1', ...
```

```
layer_names: ('conv_0', 'bn_0', 'leaky_1', 'conv_1', 'bn_1', 'leaky_2', 'conv_2', 'bn_2', 'leaky_3', 'conv_3', 'bn_3', 'leaky_4', 'shortcut_4', 'conv_5', 'bn_5', 'leaky_6', 'conv_6',
'bn_6', 'leaky_7', 'conv_7', 'bn_7', 'leaky_8', 'shortcut_8', 'conv_9', 'bn_9', 'leaky_10', 'conv_10', 'bn_10', 'leaky_11', 'shortcut_11', 'conv_12', 'bn_12', 'leaky_13', 'conv_13',
'bn_13', 'leaky_14', 'conv_14', 'bn_14', 'leaky_15', 'shortcut_15', 'conv_16', 'bn_16', 'leaky_17', 'conv_17', 'bn_17', 'leaky_18', 'shortcut_18', 'conv_19', 'bn_19', 'leaky_20',
'conv_20', 'bn_20', 'leaky_21', 'shortcut_21', 'conv_22', 'bn_22', 'leaky_23', 'conv_23', 'bn_23', 'leaky_24', 'shortcut_24', 'conv_25', 'bn_25', 'leaky_26', 'conv_26', 'bn_26',
'leaky_27', 'shortcut_27', 'conv_28', 'bn_28', 'leaky_29', 'conv_29', 'bn_29', 'leaky_30', 'shortcut_30', 'conv_31', 'bn_31', 'leaky_32', 'conv_32', 'bn_32', 'leaky_33',
'shortcut_33', 'conv_34', 'bn_34', 'leaky_35', 'conv_35', 'bn_35', 'leaky_36', 'shortcut_36', 'conv_37', 'bn_37', 'leaky_38', 'conv_38', 'bn_38', 'leaky_39', 'conv_39', 'bn_39',
'leaky_40', 'shortcut_40', 'conv_41', 'bn_41', 'leaky_42', 'conv_42', 'bn_42', 'leaky_43', 'shortcut_43', 'conv_44', 'bn_44', 'leaky_45', 'conv_45', 'bn_45', 'leaky_46',
'shortcut_46', 'conv_47', 'bn_47', 'leaky_48', 'conv_48', 'bn_48', 'leaky_49', 'shortcut_49', 'conv_50', 'bn_50', 'leaky_51', 'conv_51', 'bn_51', 'leaky_52', 'shortcut_52', 'conv_53',
'bn_53', 'leaky_54', 'conv_54', 'bn_54', 'leaky_55', 'shortcut_55', 'conv_56', 'bn_56', 'leaky_57', 'conv_57', 'bn_57', 'leaky_58', 'shortcut_58', 'conv_59', 'bn_59', 'leaky_60',
'conv_60', 'bn_60', 'leaky_61', 'shortcut_61', 'conv_62', 'bn_62', 'leaky_63', 'conv_63', 'bn_63', 'leaky_64', 'conv_64', 'bn_64', 'leaky_65', 'shortcut_65', 'conv_66', 'bn_66',
'leaky_67', 'conv_67', 'bn_67', 'leaky_68', 'shortcut_68', 'conv_69', 'bn_69', 'leaky_70', 'conv_70', 'bn_70', 'leaky_71', 'shortcut_71', 'conv_72', 'bn_72', 'leaky_73', 'conv_73',
'bn_73', 'leaky_74', 'shortcut_74', 'conv_75', 'bn_75', 'leaky_76', 'conv_76', 'bn_76', 'leaky_77', 'conv_77', 'bn_77', 'leaky_78', 'conv_78', 'bn_78', 'leaky_79', 'conv_79', 'bn_79',
'leaky_80', 'conv_80', 'bn_80', 'leaky_81', 'conv_81', 'permute_82', 'yolo_82', 'identity_83', 'conv_84', 'bn_84', 'leaky_85', 'upsample_85', 'concat_86', 'conv_87', 'bn_87',
'leaky_88', 'conv_88', 'bn_88', 'leaky_89', 'conv_89', 'bn_89', 'leaky_90', 'conv_90', 'bn_90', 'leaky_91', 'conv_91', 'bn_91', 'leaky_92', 'conv_92', 'bn_92', 'leaky_93', 'conv_93',
'permute_94', 'yolo_94', 'identity_95', 'conv_96', 'bn_96', 'leaky_97', 'upsample_97', 'concat_98', 'conv_99', 'bn_99', 'leaky_100', 'conv_100', 'bn_100', 'leaky_101', 'conv_101',
'bn_101', 'leaky_102', 'conv_102', 'bn_102', 'leaky_103', 'conv_103', 'bn_103', 'leaky_104', 'conv_104', 'bn_104', 'leaky_105', 'conv_105', 'permute_106', 'yolo_106')
```

■ YOLOv3 인공신경망 네트워크 레이어 중 출력 레이어를 확인

- 🔄 getUnconnectedOutLayers() 함수는 인공신경망 네트워크 레이어 중 출력 레이어의 인덱스를 가져옴
- ➡ YOLOv3 모델은 “yolo_82”, “yolo_94”, “yolo_106” 계층인 것을 알 수 있음

```
1 # 인공신경망 네트워크 레이어 중 출력 레이어의 인덱스를 가져온다.
2 output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
3
4 print("net.getUnconnectedOutLayers() len :", len(net.getUnconnectedOutLayers())) # net.getUnconnectedOutLayers() len : 3
5 print(net.getUnconnectedOutLayers()) # [200 227 254]
6 print(output_layers) # ['yolo_82', 'yolo_94', 'yolo_106']
```

■ 원본 비디오 파일명, 객체 검출 결과 비디오 출력 파일 이름을 설정

🔄 여기서 출력 파일은 “yolov3_street_video_output.mp4”로 설정



```
1 # 검출(Detection) 할 원본 비디오, 출력 파일
2 file_name = 'gdrive/My Drive/CV/Object Detection_YOLO/video/street.mp4'
3 output_name = 'yolov3_street_video_output.mp4' # Detection 된 output 비디오
4 elapsed_time = 0                               # 총 경과시간 초기화
```

■ 비디오에서 비디오 스트리밍을 읽어서 사물 검출 및 결과를 출력 파일로 저장

```
1 # 원본 비디오에서 video stream을 읽고, 프레임 단위로 객체 검출을 수행한다.
2
3 cap = cv2.VideoCapture(file_name)
4 writer = None
5 if not cap.isOpened():
6     print('--(!)Error opening video capture')
7     exit(0)
8 while True:
9     ret, frame = cap.read()
10    if frame is None:
11        # close the video file pointers
12        cap.release()
13        # close the writer point
14        writer.release()
15        print('--(!) No captured frame -- Break!')
16        print("elapsed time {:.3f} seconds".format(elapsed_time))
17        break
18    detectAndDisplay(frame)
```

```
=== A frame took 0.857 seconds
4 car: 95.66
7 car: 87.11
10 car: 92.51
14 car: 71.91
16 car: 95.42
18 car: 91.22
22 car: 84.95
24 car: 93.88
25 person: 62.04
26 person: 93.56
=== A frame took 0.885 seconds
--(!) No captured frame -- Break!
elapsed time 447.454 seconds
```

수행 결과 447.454 초 소요

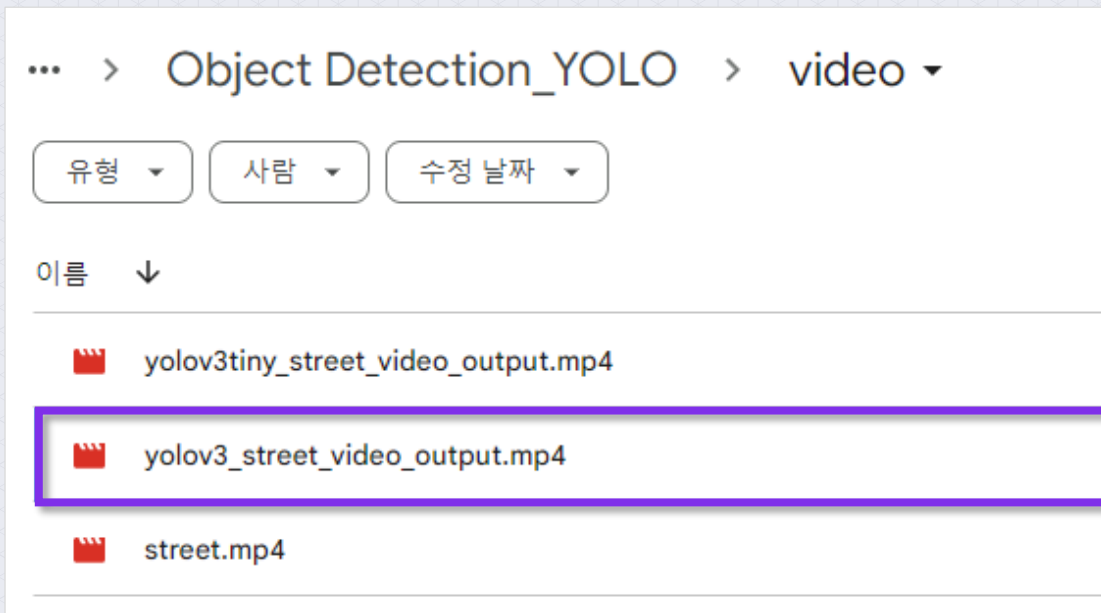
■ 실행 결과 VM에 'yolov3_street_video_output.mp4' 파일이 생성된 것을 확인

```
# 실행 결과 'yolov3_street_video_output.mp4' 파일 확인  
  
!!ls  
  
gdrive sample_data yolov3_street_video_output.mp4
```

■ VM에 'yolov3_street_video_output.mp4' 파일을 내 구글 드라이브로 이동

```
# 내 구글 드라이브로 파일 이동  
  
!mv yolov3_street_video_output.mp4 gdrive/My# Drive/CV/Object# Detection_YOLO/video/  
print('yolov3_street_video_output.mp4 move complete!!')  
  
yolov3_street_video_output.mp4 move complete!!
```

■ 결과 비디오 출력



■ 결과 비디오 출력



YOLOv3-tiny.weights, YOLOv3-tiny.cfg

■ YOLOv3-tiny.weights, YOLOv3-tiny.cfg 파일을 불러와 YOLO 네트워크를 연결

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨

```
1 # Load YOLOv3-tiny
2 # 미리 학습된 딥러닝 파일을 OpenCV DNN 모듈로 실행
3 # 순전파(추론)만 가능하며 학습은 지원하지 않는다.
4
5 net = cv2.dnn.readNet("gdrive/My Drive/CV/Object Detection_YOLO/config/yolov3-tiny.weights",
6                       "gdrive/My Drive/CV/Object Detection_YOLO/config/yolov3-tiny.cfg")
```

■ YOLOv3 네트워크의 모든 계층 이름을 출력

🔄 getLayerNames() 함수는 네트워크의 모든 레이어 이름을 가져옴

» YOLOv3-tiny 모델은 48개 계층으로 구성된 것을 알 수 있음

```
1  # YOLO 인공지능망 계층 이름을 출력
2
3  layer_names = net.getLayerNames()
4  print("layer_names len:", len(layer_names))    # layer_names len: 48
5  print("layer_names:", layer_names)            # layer_names: ('conv_0', 'bn_0', 'leaky_1', ...
```

```
layer_names: ('conv_0', 'bn_0', 'leaky_1', 'pool_1', 'conv_2', 'bn_2', 'leaky_3', 'pool_3', 'conv_4',
'bn_4', 'leaky_5', 'pool_5', 'conv_6', 'bn_6', 'leaky_7', 'pool_7', 'conv_8', 'bn_8', 'leaky_9', 'pool_9',
'conv_10', 'bn_10', 'leaky_11', 'pool_11', 'conv_12', 'bn_12', 'leaky_13', 'conv_13', 'bn_13', 'leaky_14',
'conv_14', 'bn_14', 'leaky_15', 'conv_15', 'permute_16', 'yolo_16', 'identity_17', 'conv_18', 'bn_18',
'leaky_19', 'upsample_19', 'concat_20', 'conv_21', 'bn_21', 'leaky_22', 'conv_22', 'permute_23',
'yolo_23')
```

■ YOLOv3-tiny 인공지능망 네트워크 레이어 중 출력 레이어를 확인

- 🔄 getUnconnectedOutLayers() 함수는 인공지능망 네트워크 레이어 중 출력 레이어의 인덱스를 가져옴
- YOLOv3-tiny 모델은 “yolo_16”, “yolo_23” 계층인 것을 알 수 있음

```
1 # 인공지능망 네트워크 레이어 중 출력 레이어의 인덱스를 가져온다.
2 output_layers = [layer_names[i - 1] for i in net.getUnconnectedOutLayers()]
3
4 print("net.getUnconnectedOutLayers() len :", len(net.getUnconnectedOutLayers())) # layer_name: ('conv_0', 'bn_0', 'leaky_1', 'pool_1', 'conv_2', 'bn_2',
5 print(net.getUnconnectedOutLayers()) # [36 48]
6 print(output_layers) # ['yolo_16', 'yolo_23']
```

■ 원본 비디오 파일명, 객체 검출 결과 비디오 출력 파일 이름을 설정

🔄 여기서 출력 파일은 “yolov3tiny_street_video_output.mp4”로 설정

```
1 # 검출(Detection) 할 원본 동영상
2 file_name = 'gdrive/My Drive/CV/Object Detection_YOLO/video/street.mp4'
3
4 output_name = 'yolov3tiny_street_video_output.mp4' # Detection 된 output 동영상
5 elapsed_time = 0 # 총 경과시간 초기화
```

■ 비디오에서 비디오 스트리밍을 읽어서 사물 검출 및 결과를 출력 파일로 저장

```
1 # 원본 비디오에서 video stream을 읽고, 프레임 단위로 객체 검출을 수행한다.  
2  
3 cap = cv2.VideoCapture(file_name)  
4 writer = None  
5 if not cap.isOpened:  
6     print('--(!)Error opening video capture')  
7     exit(0)  
8 while True:  
9     ret, frame = cap.read()  
10    if frame is None:  
11        # close the video file pointers  
12        cap.release()  
13        # close the writer point  
14        writer.release()  
15        print('--(!) No captured frame -- Break!')  
16        print("elapsed time {:.3f} seconds".format(elapsed_time))  
17        break  
18    detectAndDisplay(frame)
```

```
=== A frame took 0.204 seconds  
0 car: 66.47  
2 car: 86.15  
3 car: 57.23  
4 car: 57.35  
5 car: 70.52  
6 car: 85.00  
8 car: 50.29  
=== A frame took 0.198 seconds  
--(!) No captured frame -- Break!  
elapsed time 66.071 seconds
```

수행 결과 66.071 초 소요

■ 실행 결과 VM에 'yolov3tiny_street_video_output.mp4' 파일이 생성된 것을 확인

```
1 # 실행 결과 'yolov3tiny_street_video_output.mp4' 파일 확인
2
3 !ls
```

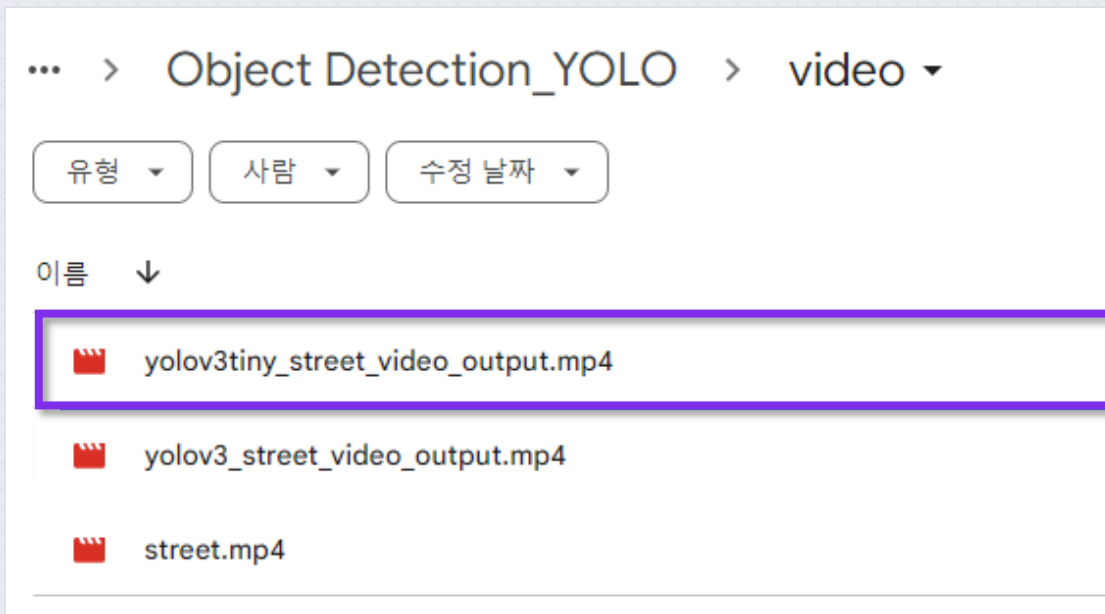
gdrive sample_data yolov3tiny_street_video_output.mp4

■ VM에 'yolov3tiny_street_video_output.mp4' 파일을 내 구글 드라이브로 이동

```
1 # 내 구글 드라이브로 파일 이동
2
3 !mv yolov3tiny_street_video_output.mp4 gdrive/My Drive/CV/Object Detection_YOLO/video/
4 print('yolov3tiny_street_video_output.mp4 move complete!!')
```

yolov3tiny_street_video_output.mp4 move complete!!

■ 결과 비디오 출력



■ 결과 비디오 출력



참고

OpenCV DNN(Deep Neural Network) 모듈

- ☞ 미리 학습된 딥러닝 파일을 OpenCV DNN 모듈로 실행할 수 있음
- ☞ 순전파(foward), 추론(inference)만 가능하며 학습은 지원하지 않음
- ☞ OpenCV 3.3 버전부터 기본 기능으로 제공
- ☞ OpenCV 4.3 버전부터 GPU(CUDA) 지원(소스 코드 직접 빌드 필요)함
- ☞ 지원하는 딥러닝 프레임워크

- Caffe
- Tensorflow
- Torch
- Darknet
- ONNX

참고

OpenCV DNN 모듈의 readNet() : 네트워크 불러오기

```
cv2.dnn.readNet(model, config=None, framework=None) # -> retval
```

- 🔄 model : 훈련된 가중치를 저장하고 있는 이진 파일 이름
- 🔄 config : 네트워크 구성을 저장하고 있는 텍스트 파일 이름
- 🔄 framework : 명시적인 딥러닝 프레임워크 이름
- 🔄 retval : cv2.dnn_Net 클래스 객체

딥러닝 프레임워크	Model 파일 확장자	Config 파일 확장자	Framework 문자열
카페	*.caffemodel	*.prototxt	"caffe"
텐서플로우	*.pb	*.pbtxt	"tensorflow"
토치	*.t7 또는 *.net		"torch"
다크넷	*.weights	*.cfg	"darknet"
DLDT	*.bin	*.xml	"dldt"
ONNX	*.onnx		"onnx"

참고

OpenCV DNN 모듈의 blobFromImage()

: 네트워크 입력 블롭(blob) 만들기, 영상의 전처리를 수행

```
cv2.dnn.blobFromImage(image, scalefactor=None, size=None,  
mean=None, swapRB=None, crop=None, ddepth=None) -> retval
```

- image : 입력 영상
- scalefactor : 입력 영상 픽셀 값에 곱할 값. 기본값은 1
- size : 출력 영상의 크기. 기본값은 (0, 0)
- mean : 입력 영상 각 채널에서 뺄 평균 값. 기본값은 (0, 0, 0, 0)
- swapRB : R과 B 채널을 서로 바꿀 것인지를 결정하는 플래그. 기본값은 False
- crop : 크롭(crop) 수행 여부. 기본값은 False
- ddepth : 출력 블롭의 깊이. CV_32F 또는 CV_8U. 기본값은 CV_32F
- retval : 영상으로부터 구한 블롭 객체

```
numpy.ndarray.shape=(N,C,H,W).dtype=numpy.float32.
```

참고 OpenCV DNN 모듈의 setInput() : 네트워크 입력 설정하기

```
cv2.dnn_Net.setInput(blob, name=None, scalefactor=None, mean=None) # -> None
```

- ↻ blob : 블롭 객체
- ↻ name : 입력 레이어 이름
- ↻ scalefactor : 추가적으로 픽셀 값에 곱할 값
- ↻ mean : 추가적으로 픽셀 값에서 뺄 평균 값

참고 (참고) OpenCV DNN 모듈의 forward() : 네트워크 순방향 실행(추론)

```
cv2.dnn_Net.forward(outputName=None) # -> retval  
cv2.dnn_Net.forward(outputNames=None, outputBlobs=None) # -> outputBlobs
```

- ↻ outputName : 출력 레이어 이름
- ↻ retval : 지정한 레이어의 출력 블롭. 네트워크마다 다르게 결정됨
- ↻ outputNames : 출력 레이어 이름 리스트
- ↻ outputBlobs : 지정한 레이어의 출력 블롭 리스트

참고

NMS(Non-Maximum Suppression, 비최대 억제)

- 비최대 억제 알고리즘을 적용해 주위 바운딩 박스에 비해 최대를 유지한 것의 인덱스 확인
- 즉 NMS를 사용하면 사물이 속한 모든 후보 중에서 최적의 셀을 식별할 수 있음

NMS는 이미지의 동일한 개체에 대한 상자 중 가장 높은 확률을 가진 상자를 선택함